

CSS4005 — Cloud Infrastructure Construction

Lecture 5: AWS Storage (EBS and S3)

Nauruzbayeva Farikha · Lesson 5



Recap — Lesson 4 (AWS Networking: VPC)

- What makes a subnet "public"? (its route table has a `0.0.0.0/0` route to an Internet Gateway)
- A private App server needs to download updates but must not accept connections from the internet. What do you add? (a NAT Gateway in a public subnet + a `0.0.0.0/0 → NAT` route in the private route table)
- How do you allow DB:3306 only from the App tier? (SG-DB inbound rule: TCP 3306, source = SG-APP — reference the security group, not an IP)

Today's agenda

- ☐ Three kinds of storage: block, file, object
- ☐ EBS: volumes, types, attach → format → mount
- ☐ EBS snapshots & encryption
- ☐ S3: buckets, objects, keys
- ☐ S3: storage classes, lifecycle, versioning
- ☐ S3: public access, website hosting, presigned URLs
- ☐ Choosing storage + not paying for forgotten resources
- ☐ Live demo → straight into today's practice

The scenario

Last week you built the three-tier VPC. Your EC2 web server is running. Users start uploading profile photos — you save them to the instance's disk. One day someone terminates the instance to "resize it", and every photo is gone. Another day a bug overwrites half the files, and there is no older copy anywhere.

Today is about where data should live in AWS so that it survives instances, mistakes, and the bill.

Block 1

Block, file, and object storage

Three ways to store bytes

	Block	File	Object
You see	A raw disk	A shared folder	Objects behind an HTTP API
Accessed via	OS mounts it, formats it	NFS / SMB mount	GET / PUT over HTTPS
Attach to	One instance (usually)	Many instances	Anyone with permission
AWS service	EBS	EFS	S3
Linux analog	/dev/sdb	NFS share	— (a web API, not a disk)

The question is never "which is best" — it's "which shape does this data have, and who needs to read it?"

Block storage in one sentence

- AWS hands your instance an **empty disk** — a sequence of blocks with no filesystem on it.
- Your OS does the rest: partition (optional), `mkfs` , `mount` , `/etc/fstab` .
- Exactly what you'd do with a new SSD in a physical server — the Linux skills from week 2 apply unchanged.

Databases, OS boot disks, anything that needs a real filesystem with low latency → block storage.

Object storage in one sentence

- You store **objects** (a file's bytes + metadata) under a **key**, in a **bucket**.
- No mounting, no filesystem, no partial in-place edits — you `PUT` a whole object, you `GET` a whole object (or a byte range).
- Practically unlimited capacity, reachable over HTTPS from anywhere you allow.

User uploads, backups, logs, static website files, build artifacts → object storage.

Block 2

Amazon EBS — Elastic Block Store

Instance store vs EBS

	Instance store	EBS volume
Physically	Disk inside the host machine	Network-attached volume
Survives reboot	Yes	Yes
Survives stop / terminate	No — data is lost	Yes (unless deleted on termination)
Available on	Only some instance types	Every instance type
Good for	Caches, scratch space, temp data	Boot disks, databases, app data

Instance store is fast but **ephemeral**. Never keep the only copy of anything important there.

An EBS volume lives in one AZ

- A volume is created **in a specific Availability Zone** and is replicated inside that AZ to protect against a single hardware failure.
- It can only be attached to an instance **in the same AZ**.
- Instance in `eu-central-1a` , volume in `eu-central-1b` → you simply can't attach it.
- Moving data to another AZ = snapshot → new volume in the other AZ (Block 3).

Same idea as last week's subnets: a subnet is AZ-scoped, and so is a volume.

EBS volume types — SSD

Type	What it's for	Key property
gp3 (General Purpose SSD)	Default for almost everything: boot disks, web servers, dev/test, small DBs	Baseline 3,000 IOPS + 125 MiB/s at any size; IOPS and throughput can be raised separately from size
gp2 (older)	Legacy default	IOPS scale with size — prefer gp3 for new volumes
io2 Block Express (Provisioned IOPS SSD)	Large, latency-sensitive databases	Highest IOPS you provision explicitly; designed for 99.999% durability; supports Multi-Attach

IOPS = small random reads/writes per second (databases care). Throughput = MiB per second (big sequential reads care).

EBS volume types — HDD

Type	What it's for	Key property
st1 (Throughput Optimized HDD)	Big sequential workloads: log processing, data warehouses, streaming	Cheap per GB, good MiB/s, poor random IOPS
sc1 (Cold HDD)	Rarely accessed large data	Lowest cost per GB of all EBS types

- st1 and sc1 **cannot be boot volumes**.
- If you're unsure → **gp3**. Change type later with Elastic Volumes — no detach needed.

Quick check

Your instance runs in `eu-central-1a` . You create a 20 GiB gp3 volume in `eu-central-1b` . You click "Attach". What happens, and how do you fix it?

The instance doesn't show up in the list — EBS volumes attach only within their own AZ. Delete it and create it in `1a` , or snapshot it and restore the snapshot into `1a` .

Attach → format → mount

```
aws ec2 attach-volume --volume-id vol-0abc --instance-id i-0123 --device /dev/sdf
# on the instance:
lsblk                                # new disk appears, e.g. nvme1n1, no mountpoint
sudo file -s /dev/nvme1n1            # "data" → empty, no filesystem yet
sudo mkfs -t xfs /dev/nvme1n1        # ONLY on a new, empty volume
sudo mkdir /data && sudo mount /dev/nvme1n1 /data
df -h /data                          # mounted and usable
```

On modern (Nitro) instances a volume attached as `/dev/sdf` shows up as `/dev/nvme1n1`. Always check `lsblk` — never guess the device name.

Persist the mount: `/etc/fstab`

```
sudo blkid /dev/nvme1n1  
# /dev/nvme1n1: UUID="4f2c...e91" TYPE="xfs"
```

```
# /etc/fstab  
UUID=4f2c...e91 /data xfs defaults,nofail 0 2
```

- Use the **UUID**, not `/dev/nvme1n1` — NVMe device names can change between boots.
- `nofail` = boot anyway if the volume is missing. Without it, a detached volume can leave the instance stuck at boot, unreachable over SSH.
- Test before rebooting: `sudo umount /data && sudo mount -a .`

mkfs destroys data

- `mkfs` writes a **new, empty** filesystem. Run it on a volume restored from a snapshot and you've wiped the restore.
- Rule: run `sudo file -s /dev/<dev>` first. Output `data` → empty, format it. Output mentions `XFS / ext4` → it already has data, **just mount**.

Formatting the wrong disk is the EBS equivalent of `down -v` : no prompt, no undo.

Delete on termination

- The **root** volume created with an instance: `DeleteOnTermination = true` by default → terminate the instance, the root disk goes too.
- **Additional** volumes you attach: `false` by default → terminate the instance, the volume keeps existing (status `available`).
- An `available` volume is attached to nothing but is still **billed per GB-month provisioned**.

Every semester someone's credits run out because of forgotten `available` volumes. Check EC2 → Volumes at the end of every lab.

Block 3

Snapshots & encryption

EBS snapshots

- A **snapshot** is a point-in-time backup of a volume.
- **Incremental**: the first snapshot copies all used blocks; later ones store only the blocks that changed since the previous snapshot.
- Stored durably in S3 by AWS — you don't see them in your buckets, you manage them under EC2 → Snapshots.
- Snapshots are **regional**: restore one to a new volume in **any AZ** of that region, or copy it to another region.

Snapshot → restore

```
aws ec2 create-snapshot --volume-id vol-0abc --description "before upgrade"
aws ec2 wait snapshot-completed --snapshot-ids snap-0def
aws ec2 create-volume --snapshot-id snap-0def \
    --availability-zone eu-central-1b --volume-type gp3
# attach the new volume → mount it (do NOT mkfs – it already has your data)
```

- Snapshot → volume in another AZ is how you **move** block data between AZs.
- For consistent DB snapshots: flush/stop writes first, or use the database's own backup tools.

EBS encryption

- Encrypted with a **KMS** key (AWS-managed `aws/ebs` key by default).
- Covers: data at rest on the volume, data moving between instance and volume, and every snapshot made from it.
- Transparent to the OS — no code changes, mount it as usual.
- **Encryption by default** is a per-region account setting: turn it on once and every new volume is encrypted.
- An existing unencrypted volume can't be "switched on": snapshot it → copy the snapshot with encryption → create a volume from the copy.

EFS — for contrast

- **Amazon EFS** = managed **NFS** file system.
- Mounted by **many** Linux instances at once, across **multiple AZs**.
- Grows and shrinks automatically — billed for what you store, not a pre-provisioned size.
- Use it when several servers need the **same files** (shared uploads, shared config). Use EBS when one server needs its own fast disk.

Quick check

Your database volume is in `eu-central-1a` . You need a copy of that data on a new instance in `eu-central-1c` for testing. What are the steps?

Snapshot the volume → create a new volume from the snapshot in `1c` → attach it to the test instance → mount it (no `mkfs`).

Block 4

Amazon S3 — Simple Storage Service

Buckets, objects, keys

- **Bucket** — a container for objects. The name is **globally unique** across all AWS accounts (3–63 chars, lowercase, digits, `-`, `.`).
- A bucket **lives in one region** you choose — the data stays there unless you replicate it.
- **Object** — the data (up to 5 TB) + metadata, identified by a **key**.
- Designed for 11 nines (99.999999999%) of durability; strongly consistent read-after-write.

```
s3://narxoz-photos-farikha/users/42/avatar.png  
  |____ bucket ____| |____ key ____|
```

"Folders" are just key prefixes

- S3 has a **flat** namespace. `users/42/avatar.png` is one key with slashes in it — not a file inside directories.
- The console *draws* folders by grouping keys on `/`.
- Prefixes still matter: lifecycle rules, permissions, and listings can target a prefix like `logs/`.

```
aws s3 ls s3://narxoz-photos-farikha/users/42/  
aws s3 cp avatar.png s3://narxoz-photos-farikha/users/42/avatar.png  
aws s3 sync ./site s3://narxoz-site-farikha/
```

S3 storage classes

Class	Use for	Trade-off
Standard	Frequently accessed data (default)	Highest storage price, no retrieval fee
Intelligent-Tiering	Unknown or changing access patterns	Moves objects between tiers automatically; small monitoring fee
Standard-IA	Infrequent access, needs fast retrieval	Cheaper storage, per-GB retrieval fee, 30-day minimum
One Zone-IA	Re-creatable infrequent data	Stored in a single AZ — lost if that AZ is lost
Glacier Instant / Flexible / Deep Archive	Archives, compliance, long-term backups	Cheapest storage; slower and/or paid retrieval, 90–180 day minimums

Exact prices differ per region and change — use the AWS Pricing Calculator, not memory.

Lifecycle rules — let S3 move data for you

```
{
  "Rules": [{
    "ID": "logs-to-ia-then-expire",
    "Filter": { "Prefix": "logs/" },
    "Status": "Enabled",
    "Transitions": [{ "Days": 30, "StorageClass": "STANDARD_IA" }],
    "Expiration": { "Days": 365 }
  }]
}
```

- **Transition:** after N days, move objects to a cheaper class.
- **Expiration:** after N days, delete them.
- Set it once, never manually clean up old logs again.

Versioning

- With versioning **enabled**, every overwrite keeps the previous object as an older **version** with its own version ID.
- A `DELETE` doesn't erase data — it adds a **delete marker**. Remove the marker and the object is back.
- Once enabled, versioning can be **suspended** but never fully turned off.
- Old versions are billed too → pair versioning with a `NoncurrentVersionExpiration` lifecycle rule.

Versioning is your protection against "a bug overwrote half the files" from the scenario.

Quick check

Versioning is enabled. Someone runs `aws s3 rm s3://bucket/report.pdf`. Is the file gone?

No — S3 added a delete marker on top. The previous version still exists; delete the marker (or copy the old version back) to restore it.

Block 5

Who can read your bucket?

Private by default

- New buckets are **private**: only identities in your account with permission can read them.
- **Block Public Access** (BPA) is **on by default** for new buckets — four switches that override any policy or ACL trying to make data public.
- New objects are **encrypted at rest by default** (SSE-S3).

Most "S3 data leak" headlines are someone turning BPA off and attaching a public policy to the wrong bucket. Leave BPA on unless you have a specific, reviewed reason.

Bucket policies — preview

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::111122223333:role/app-server" },
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::narxoz-photos-farikha/users/*"
  }]
}
```

- A JSON document attached to the bucket: **who** (Principal) can do **what** (Action) on **which objects** (Resource).
- Same policy language as IAM — **next week's** whole topic.

Static website hosting

- S3 can serve a static site (`index.html` , CSS, JS, images) from a **website endpoint**.
- Requires: enable website hosting, set an index document, and allow public read (BPA off for that bucket + a public-read policy).
- The S3 website endpoint is **HTTP only**. For HTTPS and a custom domain, put **CloudFront** in front — and then the bucket can stay private.

Fine for a lab demo or a portfolio page. For anything real: CloudFront + private bucket.

Presigned URLs — temporary access

```
aws s3 presign s3://narxoz-photos-farikha/users/42/avatar.png --expires-in 300  
# https://narxoz-photos-farikha.s3.eu-central-1.amazonaws.com/users/42/avatar.png?X-Amz-...
```

- A URL that carries a signature made with **your** credentials, valid for a limited time.
- Anyone holding it can `GET` that one object until it expires — the bucket stays private.
- Typical use: the app generates a short-lived download link for a logged-in user.
- Maximum lifetime is 7 days, and it stops working early if the signing credentials expire.

Block 6

Choosing storage & cleaning up

Which storage for which workload?

Workload	Choice	Why
OS boot disk	EBS gp3	Block device, survives stop
PostgreSQL on EC2	EBS gp3 (io2 if very IOPS-heavy)	Low-latency block storage
Shared uploads for 3 web servers	EFS — or better, S3	Many readers/writers
User photos, backups	S3 Standard + versioning	Durable, cheap, HTTP-accessible
Logs kept 1 year	S3 + lifecycle → IA → expire	Pay less as data cools
Temporary scratch / cache	Instance store	Fast, and losing it is OK

How storage is billed

- **EBS**: per GB-month **provisioned** — a 100 GiB volume with 1 GiB of data is billed as 100 GiB. Plus snapshot storage. Plus extra IOPS / throughput above gp3 baseline.
- **S3**: per GB-month **actually stored** (every version counts) + per request + data transferred out to the internet.
- **EFS**: per GB-month stored.

No numbers on purpose — they vary by region and change. Estimate with the AWS Pricing Calculator before you build.

End-of-lab cleanup checklist

- ☐ EC2 → Volumes: no forgotten volumes in state `available`
- ☐ EC2 → Snapshots: delete lab snapshots you don't need
- ☐ S3: empty the bucket — **including old versions and delete markers** — then delete it
- ☐ Stop or terminate lab instances

A versioned bucket that "looks empty" in the console may still hold every old version. Toggle **Show versions** before you call it clean.

Block 7

Today's practice

Today's practice — a disk and a bucket that survive mistakes

Part 1 — block storage

1. Create a volume (EBS, or a loop-device file on Linux)
2. Format it, mount it, persist it in `/etc/fstab` with `nofail`
3. Snapshot it, "lose" a file, restore from the snapshot

Part 2 — object storage

4. Bucket with versioning; overwrite a file, restore the old version
5. Lifecycle rule, Block Public Access, presigned URL

```
lsblk
sudo file -s /dev/nvme1n1
sudo mkfs -t xfs /dev/nvme1n1
sudo blkid /dev/nvme1n1
aws s3api put-bucket-versioning \
  --bucket <name> \
  --versioning-configuration Status=Enabled
aws s3 presign s3://<name>/notes.txt
```

No AWS account? Track B does the same with Linux loop devices and a local S3-compatible server. Submit `REPORT.md` with commands + screenshots.

By the end of this lesson, you should be able to

- ☐ Choose block, file, or object storage for a given workload
- ☐ Create, attach, format, and persistently mount an EBS volume in the correct AZ
- ☐ Back up and restore a volume with snapshots, and explain why they're incremental
- ☐ Use S3 versioning, lifecycle rules, Block Public Access, and presigned URLs
- ☐ Find and delete the storage resources that keep costing money

Before next lecture

- ☐ Finish today's practice and submit `REPORT.md` if you didn't finish in class
- ☐ Run the cleanup checklist — volumes, snapshots, versioned buckets
- ☐ Re-read the bucket policy JSON from today: next week we take that policy language apart

Next lecture

AWS IAM and Cloud Security — users, roles, policies, and least privilege.